

Seminarie 4:

Strukturer och I/O

Carl Johan Ljungstedt Linda Bromander Ulf Borgenstam

6 November 2002

Innehåll

- Strukturer
- I/O
- Slutsatser

Baserad på kapitel 6-7 i "The C Programming Language"

Vad är en "struct" ?

En struct är en samling variabler.

exempel:

```
struct _Person {  
    char *namn;  
    int kod;  
    int niva;  
};
```

För att skapa en variabel av typen _Person skriver man:
struct _Person <variabelnamn>;

exempel:

```
struct _Person p1 = {  
    "Kalle_Bromander", // namn  
    492,                // kod  
    9                   // behörighets nivå  
};
```

Här har vi deklarerat och tilldelat variabeln p1. Notera likheten med arrayer för tilldelningen.

exempel:

// Man kan deklarera variabler direkt efter strukturen.

```
struct _Person {  
    char *namn;  
    int kod;  
    int niva;  
} p2, p3;
```

// För att komma åt enskilda element i en stuktur använder
// man . (punkt).

// Om vi vill sätta koden i p2 till 456 skriver vi:

```
p2.kod = 456;
```

Nästade strukturer

Ibland kan man behöva lägga strukturer inuti strukturer.

exempel:

```
struct _Person {  
    char *namn;  
    struct {  
        int langd;  
        int kod;  
    } kod;  
    int niva;  
};
```

Notera att vi inte behöver ange något namn på den inre strukturen.

exempel:

// Så här tilldelar vi värden till variabler av den nya typen:

```
struct _Person p1 = {  
    "Kalle_Bromander",  
    { 4, 4928 },  
    9  
};
```

Typdefinitioner

```
typedef struct _Person Person;
```

tillåter oss att hädanefter skriva Person istället för struct _Person. Person fungerar nu som vilken annan typ som helst.

exempel:

```
Person p2 = {  
    "Ulf_Bromander",  
    { 3, 785 },  
    6  
};
```

Arrayer, funktioner och tilldelning

Vi kan skapa arrayer av strukturer, använda tilldelningsoperatoren på dem och funktioner kan dessutom ta hela strukturer som argument eller returtyp.

exempel:

```
// En array av personer
```

```
Person personal[2];
```

```
// En funktion som returnerar en struktur
```

```
Person skapaPerson(char *namn, int niva) {
```

```
    Person p;
```

```
    p.namn = namn;
```

```
    p.kod = nykod(niva);
```

```
    p.niva = niva;
```

```
    return p;
```

```
}
```

```
// Tilldelning av strukturer till strukturer
```

```
personal[0] = p1;
```

```
personal[1] = skapaPerson("Linda_Borgenstam", 10);
```

Pekare till strukturer

En pekare till en struktur skrivs som:

```
struct _Person *forsta;
```

Om vi det finns en typdefinition Person kan vi skriva:

```
Person *forsta = &p1;      // Låt forsta peka på p1
```

För att komma åt element i en struktur till vilken vi har en pekare skriver man:

```
(*forsta).namn = "Niklas_Ljungstedt";
```

eller

```
forsta->namn = "Kalle_Ljungstedt";
```


Rekursiva strukturer

Man kan använda pekare till att skapa rekursiva datatyper. Här är ett exempel på en enkel länkad lista:

```
struct _Person {  
    char *namn;  
    struct {  
        int langd;  
        int kod;  
    } kod;  
    int niva;  
    struct _Person *nasta;  
};
```

En rekursiv struktur får inte innehålla sig själv utan måste i så fall innehålla en pekare.

exempel:

```
typedef struct _Person Person;
```

```
Person *laggTillPerson(Person *lista, char *namn, int niva) {  
    Person *temp = sistalistan(lista);  
  
    temp->nasta = (Person *)malloc(sizeof(Person));  
    *temp->nasta = skapaPerson(namn,niva);  
    return nasta;  
}
```

Bit-field

Bitfält är som strukturer där elementen kan ha storlekar ner till en bit.

```
volatile struct {  
    unsigned int in : 1;  
    unsigned int inniva : 4;  
    unsigned int inlagnd : 5;  
    unsigned int inkod : 22;  
} *dorren;
```

exempel:

```
void kolladorren() {  
    while(!dorren->in) {}  
  
    if (tilltrade(dorren->inniva, dorren->inlagnd,  
                dorren->inkod)  
    {  
        oppnadorren();  
    }  
    else  
    {  
        rodlampa();  
    }  
}
```

Union

I en union mappas alla element till samma minnesadress.

```
union Test {  
    int i;  
    char c[13];  
} t;
```

exmpel:

```
/* Antag att t.c innehåller: "hello world!" */
```

```
printf(t.c);
```

```
t.i=0x20202020; /* 0x20 är mellanslag i ASCII */
```

```
printf(t.c);
```

```
/* ger utskriften: */
```

```
/* hello world! */
```

```
/*  o world! */
```

Öppna och stänga filer

```
FILE *fopen(char *filnamn, char *mode);
```

fopen öppnar en fil och returnerar en pekare till en filström.

```
int fclose(FILE *f);
```

fclose stänger filen och frigör minne.

exempel:

```
void lasafil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) { // Om fopen lyckades
        laspersonalfranfil(f);
        fclose(f);
    }
    else { // Annars...
        fprintf(stderr, "det gick inte");
    }
}
```

```
void skrivfil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) { // Om fopen lyckades
        skrivpersonaltillfil(f);
        fclose(f);
    }
    else { // Annars...
        fprintf(stderr, "det gick inte");
    }
}
```

Vad är fprintf?

```
int fprintf(FILE *f, char *format, ...);
```

output - omvandlingstecken för fprintf:

d,i	int:decimal heltal
o	int: unsigned oktav heltal
xX	int:unsigned hexadecimal heltal
u	int: unsigned decimal heltal
c	int: single character
s	char*
f	double: [-]m.d, d är given precision
e,E	double: [-]m.d e (+ -)xx , d är given precision

exempel:

```
void skrivpersonaltillfil(FILE *f) {  
    Person *p = forsta;  
    while (p != NULL) {  
        fprintf(f, "%s %i %i %i\n", p->namn,  
            p->kod.langd,  
            p->kod.kod, p->niva);  
        p = p->nasta;  
    }  
}
```

Vad är fscanf?

```
int fscanf(FILE *f, char *format, ...);
```

input - omvandlingstecken för fscanf:

d,i	decimal heltal
o	unsigned oktav heltal
x	hexadecimal heltal
u	unsigned decimal heltal
c	single character
s	char*
e, f, g	flyttal

exempel:

```
void laspersonalfranfil(FILE *f) {  
    char *namn = (char *)malloc(50);  
    int kodlangd, kod, niva;  
  
    while (fscanf(f, "%s %i %i %i", namn, &kodlangd,  
                  &kod, &niva) == 4)  
    {  
        Person *p = laggTillPerson(namn, niva);  
  
        p->kod.langd = kodlangd;  
        p->kod.kod = kod;  
    }  
}
```

Standardströmar

stdin är standard strömmen för indata (ofta tangentbordet).

stdout är standardströmmen för utdata.

stderr är standardströmmen för felmeddelanden och dyl.

Övrig I/O

```
int getc(FILE *f);  
int getchar(void);
```

```
int putc(int c, FILE *f);  
int putchar(int c);
```

```
int printf(char *format, ...);  
int scanf(char *format, ...);
```

```
exit(int e);
```

```
char *fgets(char *buffer, int maxlengd, FILE *f);  
int fputs(char *line, FILE *f);
```


Slutsatser

Strukturer ger en möjlighet att gruppera variabler och behandla dem som en enhet.

Språket C innehåller inga I/O funktioner i sig själv utan förlitar sig på ett rikligt standardbibliotek. Biblioteket innehåller en mängd funktioner för in och utmatning till filer och strömmar.